

Baseline Literature Review: Audio-JEPA for Expressive, Low-Resource Arabic TTS (KASP 2026)

TL;DR

- **Encoder side:** The single closest match is **data2vec / data2vec 2.0 for speech** (EMA teacher, latent-target masked prediction — conceptually identical to the team's design), but for an 8-week, low-compute, mel-spectrogram project the most *runnable* baseline is either the **Audio-JEPA repo** (github.com/LudovicTuncay/Audio-JEPA, MIT) — a from-scratch I-JEPA-on-mel implementation that exactly matches the proposed architecture — or **BEST-RQ in SpeechBrain (Apache-2.0)**, the most compute-efficient mel-input SSL recipe that trains to convergence quickly.
- **TTS side:** Do NOT build a novel decoder from scratch. Adopt an existing working structure — **Conformer-FastSpeech2 + HiFi-GAN in ESPnet2** (Apache-2.0) is the closest match to the proposed conformer + duration-predictor + cross-attention decoder — and add SSL conditioning; for a fast Arabic-specific starting point, fork **nipponjo/tts-arabic-pytorch** (FastPitch/Tacotron2 + HiFi-GAN, diacritized, pretrained checkpoints).
[GitHub](#)
- **Arabic reality check:** Strong open Arabic TTS baselines already exist — **ArTST** (MBZUAI, SpeechT5-based, MSA), **MMS-TTS-ara** (Meta VITS), and **ArVoice/CIArTTS corpora**. The team's defensible novelty is the **Audio-JEPA encoder + SSL-conditioned low-resource adaptation on Common Voice**, not "another Arabic TTS." Diacritization is the critical Arabic-specific risk; use a neural diacritizer (Shakkala/Farasa/CAMeL Tools) plus a phonemizer.

Key Findings

1. **There ARE two papers literally named "Audio-JEPA"/"A-JEPA," and both are directly on-point.**
 - **A-JEPA** (Fei et al., "A-JEPA: Joint-Embedding Predictive Architecture Can Listen," arXiv 2311.15830) encodes visible mel-spectrogram patches with a context encoder and predicts EMA-target-encoder representations of masked regions, using a curriculum time-frequency-aware masking scheme (annealing from random toward SpecAugment-style structured masks). **No official code or checkpoints are available** — confirmed explicitly by the Tuncay et al. paper: "no official implementation or checkpoints for A-JEPA are available." [arxiv](#) [arxiv](#)
 - **Audio-JEPA** (Tuncay, Labbé, Benetos, Pellegrini, arXiv 2507.02915, ICME 2025) is a from-scratch I-JEPA adaptation to mel-spectrograms. **Its public GitHub repo** (github.com/LudovicTuncay/Audio-JEPA) **exists and contains real, usable code** (PyTorch Lightning + Hydra, MIT license, 49 stars, 12 commits) — but pretrained checkpoints are promised in the paper ("All code and pretrained checkpoints will be released") and are NOT visibly released in the repo (only local-save-during-training is documented). Architecture: ViT-Base context & target encoders (768-dim, 12

layers, 12 heads, 16×16 patches, 85.4M params each) + a lightweight asymmetric predictor (384-dim, 6 layers, 11.3M params); 96.7M params trainable, 85.4M at inference. Mel input: 128 bands × 256 time bins, 10s clips at 32 kHz. Masking: random patches, ratio uniformly sampled 40–60%. Critically, **it found I-JEPA-style block masking worse than random masking for audio** (audio events span wide frequency bands). Pretrained on 1,921,982 AudioSet clips (~5,338 hrs) using 4× V100 for 100k steps (~13 epochs) in ~14 hours. It reports "comparable performance to wav2vec 2.0 and data2vec while using less than one-fifth of their training data," but the tables show it underperforms on fine-grained speech tasks (speaker verification, keyword spotting) — a caveat given this is a speech-TTS project.

(arxiv + 5)

2. **The closest conceptual match is data2vec (speech).** data2vec and data2vec 2.0 (Baevski et al., ICML 2022 / 2023) train a student to regress the contextualized latent representations of an EMA teacher that sees the unmasked input — exactly the team's context/target/EMA design ($\theta_{\text{teacher},i} = \gamma \cdot \theta_{\text{student},i} + (1-\gamma) \cdot \theta_{\text{teacher},i-1}$, with stop-gradient). Official code is in fairseq ([examples/data2vec](#)); it operates on raw waveform via a CNN feature extractor (not mel), and is comparatively compute-heavy (Base model: 400k updates, 63-min audio batches). The team's design differs in two ways: (a) it uses **mel-spectrogram input** (like A-JEPA/Audio-JEPA), and (b) it adds **VICReg variance-covariance regularization** for anti-collapse instead of relying on data2vec's EMA + target-normalization tricks. (Anwarvic's Blog + 3)
3. **JEPA-for-speech is an active 2025–2026 frontier**, confirming the project's novelty is real but the ground is moving: WavJEPA (raw waveform, arXiv 2509.23238), "JEPA as a Neural Tokenizer" with Density Adaptive Attention (arXiv 2512.07168, FSQ tokens + HiFi-GAN decoder at 47.5 tokens/sec), and Stem-JEPA (music). None is a drop-in speech-TTS encoder; none provides the compact, mel-input, VICReg-regularized recipe the team wants. (arXiv) (Hugging Face)
4. **Other reproducible SSL speech encoders:**
 - **HuBERT** (Hsu et al. 2021): raw waveform, offline k-means targets, iterative multi-stage; code + checkpoints in fairseq / HuggingFace ([facebook/hubert-base-ls960](#)). (arxiv)
 - **wav2vec 2.0** (Baevski et al. 2020): raw waveform, contrastive + quantizer; fairseq/HF. Time-domain span masking ($p \approx 0.065$, span $M \approx 10$, $\approx 49\%$ masked). (arxiv)
 - **WavLM** (Chen et al. 2022): HuBERT + denoising; HF ([microsoft/wavlm-base](#)). Best all-round for downstream conditioning/speaker-similarity features. (Anwarvic's Blog)
 - **BEST-RQ** (Chiu et al. 2022): **mel-filterbank input**, frozen random-projection quantizer, Conformer encoder. **Open, efficient SpeechBrain recipe (Apache-2.0)**. Per Whetten, Parcollet, Dinarelli & Estève, "Open Implementation and Study of BEST-RQ for Speech Processing" (arXiv 2405.04296, 2024), "BEST-RQ demonstrates comparable performance to wav2vec 2.0 while speeding up pre-training time by more than half"; follow-up work from the same group (arXiv 2509.12101) cites it as "2.5 times faster than wav2vec 2.0." It also avoids

representation collapse by construction (fixed quantizer). This is the most compute-friendly mel-input SSL baseline. (arXiv + 2)

5. **SSL-conditioned / low-resource TTS is well-established.** The dominant pattern: extract SSL features (HuBERT/wav2vec2/WavLM), optionally discretize via k-means into units, and drive a unit-based / FastSpeech-style TTS ("Low-Resource Self-Supervised Learning with SSL-Enhanced TTS," arXiv 2309.17020; QS-TTS, arXiv 2309.00126; "Transfer Learning Framework for Low-Resource TTS," Kim et al. 2022). DINO-VITS uses SSL speaker features for data-efficient zero-shot TTS. This literature directly supports the team's Phase-2 plan of conditioning a decoder on encoder representations. (arXiv)

(ISCA Archive)

6. **Existing Arabic TTS is mature and open:**

- **ArTST** (Toyin et al., ArabicNLP 2023; MBZUAI Speech Lab): SpeechT5-based, MSA, pretrained-from-scratch then fine-tuned for TTS/ASR/dialect-ID. TTS checkpoint on HF ((MBZUAI/speecht5_tts_clartts_ar), fine-tuned on ClArTTS), runnable via 🤗 Transformers with the (microsoft/speecht5_hifigan) vocoder. v1.5 adds diacritics (pretrained on MGB2 + Tashkeela). Code: github.com/mbzuai-nlp/ArTST. Note: the ArTST ASR checkpoints are cc-by-nc-4.0. (GitHub + 6)
- **MMS-TTS-ara** (Meta, (facebook/mms-tts-ara)): VITS end-to-end, 36.3M params, in 🤗 Transformers from v4.33. MMS weights are **CC-BY-NC 4.0**. (PromptLayer + 2)
- **nipponjo/tts-arabic-pytorch**: FastPitch/Tacotron2 + HiFi-GAN, trained on the Arabic Speech Corpus, **diacritized input, pretrained weights, ONNX package ((tts_arabic)), bundled Shakkala/Shakkalha diacritizers**. Easiest turnkey Arabic FastSpeech-family baseline (paper: "Arabic TTS with FastPitch: Reproducible Baselines, Adversarial Training, and Oversmoothing Analysis"). (GitHub) (ResearchGate)
- **Corpora**: ClArTTS (~12 hrs classical, single male, Grad-TTS/Glow-TTS baselines); **ArVoice** (Toyin, Marew, Alblooshi, Magdy & Aldarmaki, Interspeech 2025, pp. 4808-4812, doi 10.21437/Interspeech.2025-1550): "a total of 83.52 hours of speech across 11 voices; around 10 hours consist of human voices from 7 speakers," multi-speaker MSA with diacritized transcripts (partial subset on HF (MBZUAI/ArVoice), main subset gated); Arabic Speech Corpus (~3.7 hrs MSA, Buckwalter, phoneme-aligned). (Hugging Face + 4)
- **XTTS v2** (Coqui): multilingual zero-shot incl. Arabic (VQ-VAE + GPT-2 + HiFi-GAN); heavier, autoregressive. (arxiv)

7. **Common Voice Arabic caveat:** The CV 18.0 Arabic subset is ~91 hours validated (per the Open Universal Arabic ASR Leaderboard, arXiv 2412.13788, which states CV 18.0 Arabic = "91 hours of validated Arabic speech, including many common dialects such as Egyptian, Syrian, North African"). As a cross-check, Mozilla's own datasheet for a later release (Arabic v23.0) lists 158 hours recorded / 92 hours validated from 1,654 speakers — so the "~91 hours validated, multi-dialect despite MSA labeling" characterization holds. It is ASR-sourced (variable recording quality, multi-speaker) — good for SSL pretraining

(Phase 1) but suboptimal for clean single-speaker TTS (Phase 2). Pairing with ArVoice/ClArTTS for the TTS stage is advisable. (arXiv) (Mozillafoundation)

8. **Arabic-specific tooling:** Unvocalized text is the dominant failure mode. Neural diacritizers strongly outperform rule-based tools: per Fadel et al., "Arabic Text Diacritization Using Deep Neural Networks" (arXiv 1905.01965), the neural **Shakkala** system reaches "DER and WER values as low as 2.88% and 6.37%," versus the best non-neural system (**Mishkal**) at "13.78% and 21.92%." Diacritizers trained on Classical Arabic degrade on MSA. G2P/phonemization: (espeak-ng) Arabic (used by ESPnet/MMS pipelines) and CAMEL Tools; Buckwalter transliteration is a common intermediate representation. CAMEL Tools / Camelira and Farasa (QCRI, fast) are the best-maintained open Arabic NLP toolkits. (Medium + 3)

Details

Goals 1 & 2 — Encoder baseline and which to fork

The team's Phase-1 spec (mel → mask → context encoder + EMA target encoder → predictor → latent L2 loss + VICReg) is a **hybrid of I-JEPA (architecture, mel patches, asymmetric predictor) and data2vec (EMA teacher on speech, latent regression) with a VICReg anti-collapse twist**. No published model is exactly this — that gap is the project's genuine novelty. Three candidates to fork:

- **Audio-JEPA (LudovicTuncay/Audio-JEPA, MIT)**. Closest architectural match — literally I-JEPA on mel-spectrograms with an EMA target encoder and lightweight asymmetric predictor. Code exists and is runnable (PyTorch Lightning + Hydra). It does NOT include VICReg (uses L2 latent loss with stop-gradient EMA only) and ships no checkpoints, so the team would add the VICReg variance/covariance terms and pretrain themselves — a clean, well-scoped contribution. Its ViT-Base (85M) exceeds "compact," but the Hydra config is trivially shrinkable (fewer layers/dims).
- **I-JEPA official (facebookresearch/ijepa, CC BY-NC 4.0, archived)**. Cleanest reference for masking/predictor/EMA machinery, but image-only and non-commercial — use as a *reading/reference* base, not the fork.
- **data2vec 2.0 (fairseq)**. Closest training objective, but raw-waveform and heavy; better as conceptual citation than codebase.

Recommendation: Fork **Audio-JEPA** as the primary skeleton (it already implements exactly the mel + context/target-EMA + predictor pipeline the proposal describes) and add VICReg. If training stability or compute becomes a blocker, fall back to **BEST-RQ in SpeechBrain** (Apache-2.0, mel input, Conformer, fastest to converge, collapse-free by construction) as a pragmatic Plan-B encoder — it sacrifices the "latent EMA prediction" purity but is the most likely to yield usable Arabic representations within 8 weeks.

Goal 3 — SSL-conditioned, low-resource TTS baseline

The proposed decoder (cross-attention on linguistic features + conditioning c, duration predictor, conformer blocks, mel output) is **Conformer-FastSpeech2**. The reference

implementation is **ESPnet2's** `fastspeech2` with a **Conformer encoder** (Apache-2.0), exposing the exact modules the proposal names (ConformerEncoder, DurationPredictor, LengthRegulator, VariancePredictor). HuggingFace also ships `FastSpeech2Conformer`. Pair with **HiFi-GAN** vocoder (or ESPnet's joint JETS training of FastSpeech2 + HiFi-GAN). For SSL conditioning, follow the unit-based / feature-conditioned recipes (arXiv 2309.17020, QS-TTS): feed continuous encoder embeddings as an auxiliary conditioning stream, or discretize them into k-means units. `Kan-bayashi` `arxiv`

Recommendation: Use **ESPnet2 Conformer-FastSpeech2 + HiFi-GAN** as the TTS baseline (closest to the proposed architecture; mature; supports x-vector/speaker and auxiliary conditioning). For a faster Arabic-specific bootstrap with working pretrained weights, start from **nipponjo/tts-arabic-pytorch (FastPitch + HiFi-GAN)** and modify its conditioning path to accept Audio-JEPA embeddings.

Goal 4 — Existing Arabic TTS: adopt structure vs improve Arabic-specific

Verdict: Adopt an existing working full structure and apply/adapt it to Arabic — do NOT try to beat ArTST or MMS on raw Arabic TTS quality. ArTST (SpeechT5) and MMS-TTS (VITS) are strong, open, pretrained Arabic MSA systems; commercial systems (ElevenLabs) and large-scale efforts (QCRI's "More Data, Fewer Diacritics," which built ~4,000 hours of Arabic TTS training data; Fanar Aura) set a high bar. The team's realistic, defensible contribution is the **encoder (Audio-JEPA)** plus demonstrating that **SSL pretraining on unlabeled Common Voice Arabic improves a compact FastSpeech2-style TTS in low-resource conditions** — using ArTST / MMS-TTS-ara / nipponjo purely as *baselines to compare against*, not systems to reinvent. `arxiv`

Goal 5 — Arabic challenges & tooling

- **Diacritization (critical):** unvocalized MSA is pronunciation-ambiguous. Use a neural diacritizer — **Shakkala/Shakkelha** (bundled in nipponjo's repo), **Farasa** (QCRI, fast), or **CAMeL Tools / Camelira** (best-maintained open toolkit). Rule-based **Mishkal** is a weak fallback (DER $\approx 13.78\%$ vs Shakkala's 2.88%). Beware CA-trained diacritizers degrading on MSA.
- **G2P/phonemization:** `espeak-ng` Arabic (integrates with ESPnet/MMS), CAMeL Tools; Buckwalter transliteration as a robust intermediate.
- **MSA vs dialect:** Common Voice Arabic is labeled MSA but contains dialectal utterances; filter or accept as pretraining noise. Keep TTS synthesis targets MSA and diacritized where possible.

Goal 6 — Week-1 mentor questions

- **(a) Target vs context encoder training order:** Train **jointly**. In JEPA/I-JEPA/data2vec, the target encoder is NOT pretrained separately — it is an EMA copy of the context/student encoder updated as $\xi \leftarrow \tau \xi + (1-\tau) \cdot \theta$, with a stop-gradient on the target branch. Pretraining it separately would break the "moving target" mechanism that both prevents

trivial/collapsed solutions and supplies progressively richer contextualized targets. The proposal's own EMA formula confirms this is the intended design. (arxiv)

- **(b) Frozen vs fine-tuned SSL features for TTS: Start with frozen SSL features** (train the decoder on top; cheap, stable, avoids catastrophic forgetting on tiny paired data), then optionally **unfreeze with a low learning rate ($\sim 1/10$ of base)** once the decoder converges. With Common Voice's limited clean paired data, frozen-then-lightly-finetuned is the safer low-resource choice.
- **(c) Evaluation:** Combine (i) subjective **MOS** (naturalness) and, for system comparisons, **CMOS / MUSHRA**; (ii) an automatic MOS predictor (**UTMOS**) for cheap iteration; (iii) **intelligibility via WER/CER** by back-transcribing generated audio with a strong ASR — **Whisper-large-v3-turbo** is the de-facto standard in recent Arabic TTS benchmarks (e.g., Ramsa, MMS); (iv) **MCD** (with DTW alignment) for spectral fidelity; (v) **speaker similarity** via cosine of WavLM/ECAPA-TDNN embeddings; (vi) prosody metrics (F0/duration error). For an 8-week project, report **WER + UTMOS + MCD** at minimum, plus a small MOS test. (Zilliz + 2)
- **(d) Mel masking strategy:** For speech mel-spectrograms, **the Audio-JEPA finding is that random patch masking (40–60%) beats I-JEPA-style multi-block masking** because audio events span wide frequency bands. data2vec/wav2vec2 use **time-domain span masking** ($p \approx 0.065$, span $M \approx 10$, ≈ 49 – 65% masked). Recommended: start with **random/span time masking $\approx 50\%$** ; optionally add SpecAugment-style time+frequency masking. Avoid pure I-JEPA multi-block masking for audio.

Recommendations

Stage 0 (Week 1): Lock the pipeline. Clone **Audio-JEPA (MIT)** for the encoder and **ESPnet2 Conformer-FastSpeech2 + HiFi-GAN (Apache-2.0)** (or nipponjo/tts-arabic-pytorch for a faster Arabic bootstrap) for TTS. Stand up the Common Voice 18 Arabic loader $\rightarrow$ log-Mel front-end. Choose a diacritizer (CAMEL Tools or Shakkala) and (espeak-ng) G2P. Confirm the EMA/stop-gradient design (train jointly).

Stage 1 (Weeks 2–4): Pretrain a **compact** Audio-JEPA encoder (shrink ViT-Base to ~ 6 layers / 384–512 dim; random patch masking $\approx 50\%$; add VICReg variance+covariance terms with the proposal's λ_2, λ_3). Monitor for collapse via embedding variance/rank (cheap early indicators, per arXiv 2501.05966). **Fallback threshold:** if the JEPA encoder collapses or fails to beat random-init features on a linear probe by end of Week 3, switch to **BEST-RQ (SpeechBrain)** as the encoder.

Stage 2 (Weeks 4–6): Train the FastSpeech2 decoder conditioned on **frozen** encoder features, on the cleanest available paired data (ArVoice/ClArTTS for fidelity; a filtered CV subset for coverage).

Stage 3 (Weeks 6–8): Ablate frozen vs low-LR fine-tuned encoder; compare against ArTST, MMS-TTS-ara, and a no-SSL FastSpeech2 baseline. Evaluate with WER (Whisper-large-v3-turbo), UTMOS, MCD, speaker similarity, and a small MOS test. **Success threshold:** the SSL-conditioned model beats the no-SSL baseline on WER/UTMOS in the low-resource regime.

Benchmarks that change the plan: (1) If a pretrained data2vec-audio or JEPA-speech checkpoint appears on HF, prefer fine-tuning it over pretraining from scratch. (2) If clean paired Arabic data < ~5 hrs, pivot to unit-based/discrete-token TTS. (3) If compute allows > ~50 GPU-hours, scale the encoder toward the full ViT-Base config.

Caveats

- **Audio-JEPA checkpoints are not confirmed released** — the team will likely pretrain from scratch. The authors used 4× V100 for ~14 hrs on 5,338 hrs of AudioSet; Common Voice Arabic at ~91 hrs is far smaller, so a compact model is feasible on modest GPUs. [arxiv](#)
- **Licensing:** Audio-JEPA = MIT; ESPnet/SpeechBrain = Apache-2.0 (permissive). facebookresearch/ijepa, MMS, and the ArTST ASR checkpoints are **CC BY-NC 4.0 (non-commercial)** — fine for a research project, but flag if any commercialization is intended.
- **VICReg + EMA together is under-explored for speech JEPA** — the combination is a research bet, not a proven recipe; keep plain L2 + EMA (data2vec-style) as a fallback objective.
- **Common Voice ≠ clean TTS data.** Expect the SSL-pretraining benefit to show more on intelligibility/robustness than on studio-grade naturalness; pair with ArVoice/ClArTTS for the TTS stage.
- The JEPA-speech and Arabic-TTS literature is moving fast (late-2025/2026 preprints, several not yet peer-reviewed); the exact arXiv identifier and 4,000-hour figure for QCRI's "More Data, Fewer Diacritics" should be re-verified before formal citation, though the paper and its central claim are well attested.
- Audio-JEPA's "comparable to wav2vec 2.0/data2vec" claim is the authors' framing; on ~a third of X-ARES tasks (including some speech tasks) it places last, so treat it as a starting skeleton rather than a proven speech encoder. [arxiv](#)